Programowanie Obiektowe — PROI

Symulacja działania hotelu

Raport końcowy

ZESPÓŁ PROJEKTOWY

Bartosz Grzanka

Konrad Orlikowski

NUMER GRUPY

26L 201

Spis treści

1. Wprowadzenie
   1.1. Cel projektu
   1.2. Zakres funkcjonalny
   1.3. Środowisko techniczne
   1.4. Dokumentacja wstępna i uwagi prowadzącego
2. Scenariusz demonstracyjny
   2.1. Hotel Grand Aurora
   2.2. Procedura uruchomienia i odtwarzania wyników
   2.3. Oczekiwany raport końcowy i jego interpretacja
3. Realizacja założeń projektowych
   3.1. Zarządzanie czasem
   3.2. Zarządzanie pokojami
   3.3. Obsługa gości i rozliczenia
   3.4. Zarządzanie personelem — przegląd
   3.5. Szczegółowa mechanika personelu (Staff)
4. Architektura rozwiązania
   4.1. Pętla symulacji — kolejność etapów ticka
   4.2. Modularyzacja kodu
   4.3. Wzorce projektowe
5. System logowania i raportowania
   5.1. Format wpisów i kategorie zdarzeń
   5.2. Mechanizm log_every_n_ticks — warstwa konsolowa i pamięciowa
   5.3. Eksport do pliku — Logger::prepare_log
   5.4. Raport końcowy i progi oceny
6. Zdarzenia losowe i obsługa awarii
   6.1. EventGenerator — losowanie i typy zdarzeń
   6.2. actual_failure jako odpowiednik RoomBreakdown
   6.3. Zachowanie alarmu pożarowego (FireAlarm)
   6.4. Warunek krytyczny awarii i no-show meldunku
7. System obsługi gastronomicznej
8. Konfiguracja systemu
9. Weryfikacja poprawności
10. Podział prac w zespole
   10.1. Bartosz Grzanka
   10.2. Konrad Orlikowski
   10.3. Prace wspólne

11. Ograniczenia i kierunki rozwoju

Streszczenie

Raport dokumentuje realizację projektu z Programowania Obiektowego, polegającego na zbudowaniu symulacji dyskretno-czasowej działania hotelu. System modeluje przepływ czasu wirtualnego (ticki), zarządza zasobami pokojowymi, obsługę gości o zróżnicowanych profilach, koordynację personelu oraz reagowanie na zdarzenia losowe. Implementacja w języku C++ opiera się na polimorfizmie, kompozycji obiektów i konfiguracji zewnętrznej z plików tekstowych.

W ramach projektu opracowano kompletny scenariusz demonstracyjny Hotel Grand Aurora, mechanizm logowania z dwuwarstwowym throttlingiem i eksportem do pliku, raport końcowy jakości usług oraz zestaw szesnastu plików testów jednostkowych potwierdzających poprawność kluczowych modułów. Raport przedstawia zakres realizacji w odniesieniu do dokumentacji wstępnej (PROI_26L_201.4-1.pdf) oraz uwag prowadzącej, dokładną architekturę rozwiązania (w tym kolejność etapów pojedynczego ticka i cykl życia pracownika), podział prac w zespole oraz świadomie przyjęte ograniczenia względem planu wstępnego.

1. Wprowadzenie

1.1. Cel projektu

Celem projektu było zaprojektowanie i implementacja symulacji hotelu, w której użytkownik definiuje parametry środowiska (czas, personel, goście, prawdopodobieństwo zdarzeń), a system w sposób autonomiczny przeprowadza operacje recepcji, sprzątania, obsługi gastronomicznej i napraw awarii. Symulacja działa w trybie dyskretno-czasowym: pojedynczy krok (tick) odpowiada konfigurowalnemu odcinkowi czasu hotelowego, a całość przebiegu jest deterministyczna względem ziarna generatora pseudolosowego.

1.2. Zakres funkcjonalny

Zakres projektu obejmuje cztery filary wymienione w dokumentacji wstępnej:

- zarządzanie czasem — symulacja oparta na tickach z konfigurowalną rozdzielczością czasową,
- zarządzanie pokojami — hierarchia typów pokoi ze stanami zajętości, czystości i sprawności,
- obsługa gości — polimorficzne profile gości z mechaniką satysfakcji i rozliczeń,
- zarządzanie personelem — role pracownicze, harmonogramy, zmęczenie i wynagrodzenia.

Ponadto zaimplementowano trzyfazowy system posiłków, konfigurację z plików tekstowych, dwuwarstwowy system logowania oraz raport końcowy oceny jakości usług hotelu.

1.3. Środowisko techniczne

Tabela 1. Środowisko techniczne projektu

Element	Wartość
Język implementacji	C++20 lub wyższy
System budowania	CMake
Testy	Catch2 / doctest
Repozytorium	gitlab-stud.elka.pw.edu.pl/bgrzanka/proi_26l_201.4

1.4. Dokumentacja wstępna i uwagi prowadzącego

Raport końcowy został opracowany w odniesieniu do dokumentacji wstępnej zespołu (PROI_26L_201.4-1.pdf) oraz pliku „Komentarze do dokumentacji wstępnej”. Poniżej zestawiono, w jaki sposób uwagi prowadzącego zostały uwzględnione w implementacji, a następnie — które założenia z planu wstępnego zrealizowano w pełni, a które uproszczono lub odłożono.

Tabela 2. Uwagi prowadzącego i ich uwzględnienie

Uwaga prowadzącego	Reakcja zespołu w wersji końcowej

Zbyt szeroki zakres funkcji — możliwość rezygnacji z oceny hotelu lub awarii	Zachowano oba elementy w uproszczonej formie: ocena przez HotelReport (średnia satysfakcja), awarie przez flagę actual_failure i konserwatora — bez pełnej hierarchii RoomBreakdown/LostKey.
Logika symulacji nie powinna być w metodach behave() klas domenowych	Zastosowano: brak behave() w Guest; orchestracja w klasie Simulation i modułach Simulation*.cpp. Klasy Room/Staff/Guest są wywoływane z symulacji, nie sterują tickami samodzielnie.
Zakaz łączenia obiektów przez ID (podejście relacyjne)	Zastosowano odniesienia obiektowe: Bill przechowuje wskaźnik do Reservation, Hotel zarządza unique_ptr; rezerwacje i personel powiązani przez wskaźniki i kompozycję, bez pól typu guestID.

Mapowanie założeń z dokumentacji wstępnej na stan implementacji przedstawia tabela 3.

Tabela 3. Plan wstępny vs implementacja końcowa

Założenie (dokumentacja wstępna)	**Status**
Symulacja tick-based, losowy czas zadań	Zrealizowane
Cztery filary: czas, pokoje, goście, personel	Zrealizowane
Hierarchia pokoi Single/Double/Triple/Apartment	Zrealizowane
Pokoje Studio i czteroosobowe	Zastąpione Triple/Apartment
Szczegółowe udogodnienia pokoi (łazienki, Wi-Fi, balkon)	Poza zakresem wersji końcowej
Polimorfizm gości Business/VIP/Family	Zrealizowane (mnożniki satysfakcji)
Zachowania gości	Zrealizowane
System oceny hotelu (ukryta satysfakcja)	Zrealizowane (HotelReport)
Rozliczenia Reservation + Bill	Zrealizowane
Personel + konserwator + zmęczenie	Zrealizowane
Zdarzenia: PowerOutage, FireAlarm	Zrealizowane
Zdarzenia: RoomBreakdown, LostKey	Częściowo (actual_failure) / Niezrealizowane
Logger (konsola + plik)	Zrealizowane (log_event + Logger)
ConfigLoader + pliki konfiguracyjne	Zrealizowane (HotelLoader, StaffLoader)

2. Scenariusz demonstracyjny

2.1. Hotel Grand Aurora

W celu weryfikacji integracyjnej przygotowano plik konfiguracyjny config/hotel.txt opisujący hotel testowy o nazwie Hotel Grand Aurora. Scenariusz obejmuje pełny przekrój funkcjonalności systemu i stanowi rekomendowany sposób prezentacji projektu.

Tabela 4. Parametry scenariusza demonstracyjnego

Parametr	Wartość	Znaczenie
Liczba pokoi	18	Single, Double, Triple, Apartment na 3 piętrach
max_ticks	864	Około 3 dób symulacji (864 × 5 min = 4320 min = 72 h)
minutes_per_tick	5	Jeden tick = 5 minut czasu hotelowego
event_probability	0.3	0,3% szansy zdarzenia na tick (wartość ułamkowa)
date_start	2026-06-02	Wtorek — start symulacji w środku tygodnia
meal_attendance_percent	80	Udział gości w śniadaniu i kolacji
log_every_n_ticks	5	Throttling wyłącznie kategorii time na konsoli
log_file	hotel_simulation.log	Eksport pełnego logu po zakończeniu
Liczba pracowników	19	3× MaintenanceWorker, 3× Receptionist, 5× Housekeeper, 3× Chef, 5× Waiter

Konfiguracja personelu odzwierciedla realny rytm pracy hotelu: konserwatorzy pracują w trzech ośmiogodzinnych zmianach pokrywających dobę (0–8, 8–16, 16–24), co zapewnia natychmiastową dostępność przy każdej awarii. Pozostałe role mają zmiany dzienne dopasowane do roli (recepcja 12 h, pokojówki 8 h w godzinach 06–22, kuchnia 6 h wokół posiłków, kelnerzy 12 h).

2.2. Procedura uruchomienia i odtwarzania wyników

Kompletna procedura odtworzenia wyników raportu wymaga wyłącznie standardowego łańcucha narzędzi CMake. Wszystkie polecenia uruchamiane są z katalogu głównego repozytorium:

```
cmake -S . -B build
cmake --build build
ctest --test-dir build
./build/HotelApp --hotel config/hotel.txt
```

Pierwsze dwa polecenia konfigurują i kompilują projekt; ctest uruchamia komplet szesnastu plików testów jednostkowych — wszystkie powinny zakończyć się sukcesem. Ostatnie polecenie startuje symulację demonstracyjną. Dla celów prezentacji wprowadzono opóźnienie 200 ms na tick (sleep w pętli run()), co umożliwia śledzenie zdarzeń w konsoli w czasie zbliżonym do rzeczywistego.

Wybrane parametry symulacji można nadpisać z linii poleceń, bez konieczności modyfikacji pliku konfiguracyjnego:

```
./build/HotelApp --hotel config/hotel.txt --max-ticks 200 --log-every-n-ticks
1
```

Plik wynikowy hotel_simulation.log zostanie zapisany w bieżącym katalogu roboczym po zakończeniu run() i wywołaniu export_log_file().

2.3. Oczekiwany raport końcowy i jego interpretacja

Po pomyślnym przebiegu symulacji program publikuje pojedynczą linię raportu w ustalonym formacie:

```
REPORT: balance=531 guests=... avg_satisfaction=62 rating=good
```

Wynik balance=531 / rating=good osiągnięto po skompletowaniu personelu gastronomicznego (5 kelnerów, 3 kucharzy). We wcześniejszej wersji konfiguracji, z niedostateczną obsadą kelnerów, log zawierał 34 wpisy „No available waiter” w kategorii restaurant, a symulacja kończyła się wynikiem balance=-2573 / rating=poor. Różnica wynika z dwóch nakładających się czynników: bezpośredniej kary satysfakcji za każdy niedostarczony posiłek (mnożona przez wrażliwość gościa — VIP ×5) oraz utraty przychodu z opcjonalnego wyżywienia (food_included w rezerwacjach). Scenariusz pełni więc nie tylko funkcję smoke-testu, lecz także demonstruje wrażliwość modelu finansowego na braki kadrowe.

Ocena jakości (excellent / good / average / poor) wyliczana jest wyłącznie na podstawie gości faktycznie zameldowanych w trakcie symulacji — rezerwacje anulowane (przez gościa lub przez hotel) nie wpływają na średnią satysfakcji, co zapewnia miarodajność wskaźnika.

3. Realizacja założeń projektowych

3.1. Zarządzanie czasem

Silnik czasowy realizuje symulację dyskretną: każdy tick reprezentuje upływ minutes_per_tick minut czasu hotelowego. Użytkownik określa horyzont symulacji parametrem max_ticks; wartość 0 oznacza jedną dobę hotelową. Kluczowe właściwości warstwy czasu zestawiono poniżej:

- Długość zadań pracowniczych jest losowana z rozkładu jednostajnego (uniform_int_distribution) z wykorzystaniem wspólnego generatora simulation_rng, dzięki czemu cały przebieg jest deterministycznie odtwarzalny dla ustalonego ziarna.
- EventGenerator jest wywoływany w każdym ticku; event_probability jest wyrażone w setnych częściach procenta (skala 1–1000), co pozwala na precyzyjne sterowanie częstością awarii — od 0,1 % do 30 % na tick.
- System rozróżnia porę dnia oraz dzień tygodnia (0–6) — operacje planowane są zgodnie z indywidualnymi harmonogramami zmian (working_hours[7]).
- W kodzie istnieje specjalny tryb pracy całodobowej (start=0, end=0), w którym pracownik jest na zmianie 24/7. Tryb ten nie jest jednak używany w scenariuszu demonstracyjnym Hotel Grand Aurora — wszystkich trzech konserwatorów obsadzono na realnych ośmiogodzinnych zmianach (0–8, 8–16, 16–24).

3.2. Zarządzanie pokojami

Tabela 5. Hierarchia klas pokoi

Klasa	Charakterystyka
Room	Klasa abstrakcyjna — wspólny interfejs i stan (numer, piętro, stany to_clean / to_repair)
SingleRoom	Pokój jednoosobowy
DoubleRoom	Pokój dwuosobowy
TripleRoom	Pokój trzyosobowy
Apartment	Apartament — najwyższa kategoria, w pełni konfigurowalna

Każdy pokój posiada parametry lokalizacyjne (piętro, numer), wielkościowe i cenowe. Wirtualna metoda calculateNightlyPrice(guests) wylicza cenę doby w zależności od liczby gości, natomiast calculateStaffNeeded() zwraca wymagania kadrowe (głównie pokojówki) per typ pokoju. Stany pokoju kontrolowane są dwiema flagami: to_clean (wymaga sprzątania — ustawiana raz na dobę dla pokojów zajętych przez tick_mark_daily_dirty_rooms) oraz to_repair (wymaga naprawy).

3.3. Obsługa gości i rozliczenia

Tabela 6. Typy gości i mechanika satysfakcji

Klasa	Mnożnik kary	Profil

Family	×1	Goście rodzinni — najniższa wrażliwość na niedogodności
Business	×3	Goście biznesowi — standardowa wrażliwość
VIP	×5	Goście VIP — najwyższa wrażliwość

Każdy gość startuje z satysfakcją równą 100 (skala 0–100). Metoda reduce_satisfaction(points) odejmuje liczbę punktów pomnożoną przez wrażliwość typu — przykładowo niedostarczony posiłek (-2 punkty bazowe) odbiera VIP-owi 10 punktów satysfakcji, a rodzinie tylko 2. Mechanizm satysfakcji uwzględnia m.in.:

- opóźnienie obsługi posiłku z powodu braku kucharza lub kelnera (log w kategorii restaurant),
- zameldowanie do pokoju niesprzątanego (kara −2 punkty bazowe),
- opóźnione wymeldowanie (kara −1 punkt bazowy),
- przedłużające się awarie i tryb ewakuacji.

Warstwa rozliczeniowa składa się z dwóch ściśle powiązanych klas. Reservation reprezentuje umowę pomiędzy gościem a hotelem (daty, pokój, opcjonalne food_included), Bill agreguje pozycje rachunku: noclegi (calculateNightlyPrice × liczba dób) oraz, opcjonalnie, wyżywienie. Zgodnie z uwagą prowadzącego klasy te nie używają identyfikatorów relacyjnych — Bill przechowuje wskaźnik do Reservation, a Hotel zarządza wszystkimi obiektami przez std::unique_ptr.

Anulowanie rezerwacji jest rozróżnione semantycznie. Po stronie gościa: standardowe cancel() z karą finansową zależną od wyprzedzenia. Po stronie hotelu: cancel_by_hotel() wywoływane przez Simulation po przekroczeniu okna no-show (120 minut od planowanego check-in) — ustawia is_paid=true (gość nie zostaje obciążony), ale jeśli do daty check-in zostało mniej niż jeden dzień, dodatkowo redukuje średnią satysfakcję, ponieważ taka anulacja postrzegana jest jako wina hotelu.

Końcowa ocena jakości wyliczana jest w HotelReport na podstawie średniej satysfakcji gości zameldowanych. Stosowane progi: excellent ≥ 80, good ≥ 60, average ≥ 40, poor < 40.

3.4. Zarządzanie personelem — przegląd

Tabela 7. Role personelu hotelowego

Klasa	Typ zadania	Maks. zmiana	Uwagi
Receptionist	CheckIn, CheckOut	12 h	Obsługa zameldowań i wymeldowań (TaskType::CheckIn i TaskType::CheckOut)
Housekeeper	CleanRoom	8 h	Sprzątanie — zakaz zmiany nocnej (forbids_night_shift_schedule)
Chef	CookMeal	6 h	Gotowanie posiłków — batch do 10 osób, zakaz zmiany nocnej
Waiter	ServeMeal	12 h	Dostarczanie posiłków do pokoi

MaintenanceWorker	FixFailure	8 h	Naprawa actual_failure; jednorazowa opłata awaryjna 500

Pełna mechanika personelu — cykl życia w obrębie pojedynczego ticka, mechanizm przypisywania zadań, walidacja harmonogramu, naliczanie wynagrodzeń i zmęczenie — została wydzielona do osobnego podrozdziału 3.5. Receptionist obsługuje w równym stopniu zameldowania (Task::make_check_in) i wymeldowania (Task::make_check_out); to drugie zadanie nie zostało w pierwotnej wersji raportu wyróżnione, mimo że stanowi osobny TaskType::CheckOut z własną fabryką w src/staff/Task.cpp.

3.5. Szczegółowa mechanika personelu (Staff)

Zarządzanie personelem jest najbardziej rozbudowanym modułem behavioralnym projektu i wymaga szczegółowego omówienia. Sekcja opisuje hierarchię klas, cykl życia pracownika w obrębie pojedynczego ticka, algorytm przypisywania zadań, walidację harmonogramu zmian, model wynagrodzeń oraz model zmęczenia.

3.5.1. Hierarchia klas i polimorfizm

Klasa abstrakcyjna Staff (include/staff/Staff.hpp, src/staff/Staff.cpp) definiuje wspólny interfejs wirtualny, który implementuje pięć ról pochodnych: Receptionist, Housekeeper, Chef, Waiter, MaintenanceWorker. Każda z klas pochodnych nadpisuje co najmniej następujące metody:

- role() — zwraca enum StaffRole identyfikujący rolę,
- can_handle(task, weekday, hour) — predykat dostępności do konkretnego zadania w danym momencie,
- do_work(task) — logika realizacji zadania (np. CleanRoom obniża to_clean pokoju),
- determine_schedule() — generuje domyślny harmonogram zmian,
- max_shift_duration_hours() — maksymalna dopuszczalna długość zmiany (Recepcja 12 h, Pokojówka 8 h, Kucharz 6 h, Kelner 12 h, Konserwator 8 h),
- receive_bonus(task) — premia jednorazowa po zakończeniu zadania, specyficzna dla roli.

MaintenanceWorker dodatkowo nadpisuje calculate_task_payment(task) — opłata awaryjna 500 (FixFailure) plus premia ról-specyficzna. Mechanizm ten jest kluczowy dla scenariusza demonstracyjnego: każda awaria obciąża saldo hotelu kwotą 500 + bonus, niezależnie od czasu naprawy.

3.5.2. Cykl życia pracownika w jednym ticku — Staff::process_tick

Metoda Staff::process_tick(elapsed_minutes, hour, weekday) wykonywana jest dla każdego pracownika w każdym ticku, w ramach Hotel::tick(). Wewnątrz realizuje trzy główne kroki w ustalonej kolejności:

- **Naliczenie pensji godzinowej.** Wywołanie accrue_shift_pay(elapsed_minutes, hour, weekday) sumuje do pola to_be_paid kwotę (rate × elapsed_minutes) / 60, gdzie rate to

wynik shift_hourly_rate(). Krok jest wykonywany TYLKO, jeśli is_on_shift(hour, weekday) zwraca true.

- **Obsługa bieżącego zadania.** Jeśli is_busy == true, odliczane są remaining_task_minutes. W momencie zakończenia zadania wywoływane są kolejno: increase_fatigue(task.difficulty), receive_bonus(task), calculate_task_payment(task) → to_be_paid; pole is_busy ustawione zostaje na false.
- **Regeneracja poza zmianą.** Jeśli pracownik nie jest zajęty i jest poza zmianą (is_on_shift == false), wywoływane jest rest_tick(elapsed_minutes) — zmniejsza fatigue proporcjonalnie do elapsed_minutes i indywidualnego fatigue_rate.

Po zakończeniu process_tick dla wszystkich pracowników, Hotel oblicza różnicę get_to_pay() przed i po, a następnie wywołuje change_balance(-earned). Skutkuje to natychmiastowym potrąceniem wynagrodzeń z salda hotelu w tym samym ticku, w którym zostały naliczone — finanse pozostają więc spójne z bieżącym czasem symulacji bez konieczności agregowania wypłat dziennych.

3.5.3. Przypisywanie zadań — assign_first_available_staff

Metoda Hotel::assign_first_available_staff(task, weekday, hour) wybiera pracownika do realizacji zadania. Algorytm składa się z dwóch etapów: filtrowania i selekcji.

Etap 1 — Filtrowanie. Z kolekcji pracowników wybierani są ci, którzy spełniają jednocześnie wszystkie cztery warunki:

- ta sama rola (StaffRole zgodna z typem zadania — TaskType::CleanRoom → StaffRole::Housekeeper, itd.),
- !is_tired() — fatigue ściśle mniejsze niż 100,
- !get_is_busy() — pracownik nie realizuje obecnie innego zadania,
- can_handle(task, weekday, hour) — predykat ról-specyficzny, w tym is_on_shift(hour, weekday).

Etap 2 — Selekcja. Spośród kandydatów spełniających filtr wybierany jest pracownik z NAJNIŻSZYM bieżącym poziomem fatigue (std::min_element po get_fatigue()). Strategia ta równoważy obciążenie zespołu i zapobiega sytuacji, w której jeden pracownik dochodzi do progu zmęczenia 100, podczas gdy reszta jest bezczynna.

Jeśli zbiór kandydatów po filtrowaniu jest pusty, zadanie nie zostaje przypisane — w zależności od typu, Simulation odpowiednio rejestruje brak personelu (log w odpowiedniej kategorii) i obniża satysfakcję gości lub odracza zadanie na kolejny tick.

3.5.4. Harmonogram zmian — klasa Shift i walidacja

Każdy pracownik posiada tygodniowy harmonogram w postaci tablicy working_hours[7] (jeden wpis Shift na dzień tygodnia, indeksowany 0–6). Pojedyncza zmiana zawiera godzinę startu i końca w skali 0–24. Predykat is_on_shift(hour, weekday) obsługuje trzy przypadki:

- Standardowa zmiana dzienna (start < end): pracownik jest na zmianie dla hour ∈ [start, end).
- Zmiana nocna (start > end, np. 22 → 06): pracownik jest na zmianie dla hour ≥ start LUB hour < end.
- Tryb całodobowy (start == 0 && end == 0): zwraca zawsze true. Ten specjalny przypadek omija sprawdzanie max_shift_duration_hours i nie jest używany w scenariuszu demonstracyjnym.

Walidacja zmian odbywa się w store_day_shift(weekday, shift), uruchamianym podczas ładowania konfiguracji oraz przy programatycznej modyfikacji harmonogramu. Walidator wymusza cztery warunki:

- długość zmiany nie przekracza max_shift_duration_hours() roli (z wyjątkiem trybu 24/7),
- minimum 10 godzin odpoczynku od poprzedniego dnia,
- minimum 8 godzin odpoczynku do następnego dnia,
- Housekeeper i Chef: forbids_night_shift_schedule() — zmiana musi mieścić się w przedziale 06:00–22:00.

Naruszenie którejkolwiek z reguł powoduje rzucenie wyjątku już na etapie ładowania pliku konfiguracyjnego, co eliminuje klasę błędów typu „pracownik pracuje 16 godzin z rzędu" jeszcze przed startem symulacji.

3.5.5. Wynagrodzenia — stawki i naliczanie

Każda rola posiada dwa parametry stawki: salary (stawka dzienna podstawowa) oraz salary_night (dopłata nocna). Metoda shift_hourly_rate() zwraca pełną stawkę godzinową w zależności od pory dnia:

- W godzinach dziennych: zwraca salary / 24.
- W godzinach nocnych (22:00–06:00) dla pracownika na zmianie nocnej: zwraca (salary + salary_night) / 24.
- W trybie całodobowym (start=0, end=0): w godzinach nocnych również stosowana jest podwyższona stawka.

Naliczanie odbywa się per-tick w accrue_shift_pay(): kwota (rate × elapsed_minutes) / 60 jest dodawana do bufora to_be_paid. Po zakończeniu zadania bufor jest powiększany dodatkowo o task.bonus oraz role_bonus (specyficzny dla roli — np. Chef otrzymuje dodatek za każdy ugotowany posiłek). MaintenanceWorker przy zadaniu FixFailure otrzymuje jednorazowo 500 + task.bonus, co modeluje opłatę awaryjną.

3.5.6. Zmęczenie (fatigue)

Zmęczenie modelowane jest jako wartość całkowita w skali 0–100. Predykat is_tired() zwraca true dla fatigue ≥ 100; pracownik zmęczony jest wykluczany z filtra w assign_first_available_staff i nie

przyjmuje nowych zadań. Po zakończeniu każdego zadania wywoływane jest increase_fatigue(difficulty):

```
fatigue += fatigue_rate * task.difficulty;
if (fatigue > 100) fatigue = 100;
```

Regeneracja zachodzi w rest_tick(elapsed_minutes) — wywoływana, gdy pracownik nie jest zajęty i jest poza zmianą. Spadek fatigue jest proporcjonalny do fatigue_rate i elapsed_minutes. Parametr fatigue_rate jest indywidualny dla każdego pracownika i wczytywany z pliku konfiguracyjnego, co pozwala modelować różnice indywidualne (np. doświadczony konserwator męczy się wolniej).

3.5.7. Fabryki zadań — Task::make_*

Klasa Task agreguje wszystkie informacje o zleconym zadaniu: typ (TaskType), czas trwania w minutach, difficulty oraz bonus. Czas trwania jest losowany z rozkładu jednostajnego (uniform_int_distribution) przy użyciu wspólnego simulation_rng — dzięki czemu cały przebieg jest deterministyczny dla ustalonego ziarna. Fabryki statyczne w src/staff/Task.cpp tworzą zadania per typ:

Tabela 8. Fabryki zadań Task::make_* i przedziały czasu trwania

Fabryka	Typ zadania	Czas trwania [min]
make_check_in	CheckIn	1–10
make_check_out	CheckOut	1–10
make_clean_room	CleanRoom	5–25
make_fix_failure	FixFailure	5–30
make_cook_meal	CookMeal	10–25
make_serve_meal	ServeMeal	5–15

3.5.8. Skład personelu w scenariuszu demonstracyjnym

Łącznie 19 pracowników w pliku config/hotel.txt zapewnia ciągłość obsługi w cyklu trzydobowym:

- 3 × MaintenanceWorker — zmiany 0–8, 8–16, 16–24 (pokrycie 24/7 bez trybu 0–0),
- 3 × Receptionist — zmiany dzienne, zachodzące, łącznie pełna doba,
- 5 × Housekeeper — wszyscy w godzinach 06–22 (zakaz zmiany nocnej),
- 3 × Chef — zmiany wokół śniadania i kolacji (zakaz zmiany nocnej),
- 5 × Waiter — dyspozycja przy serwowaniu posiłków, zmiany 12 h.

4. Architektura rozwiązania

4.1. Pętla symulacji — kolejność etapów ticka

Klasa Simulation koordynuje przebieg symulacji. Pojedynczy tick wykonuje dwanaście etapów w ściśle ustalonej kolejności, zaimplementowanych jako oddzielne metody prywatne tick_*. Kolejność etapów jest kluczowa — odpowiada za to, by stan świata przed planowaniem zadań był spójny (np. naprawy rozliczone, awarie wygenerowane, anulowania przetworzone).

Tabela 9. Etapy metody Simulation::tick — kolejność wywołań

#	Etap (metoda)	Funkcja w obrębie ticka
1	tick_advance_hotel_clock	Inkrementacja current_tick_, Hotel::tick(minutesPerTick) — przesunięcie zegara wirtualnego.
2	tick_resolve_maintenance	Rozliczenie ukończonych napraw: actual_failure = false, czyszczenie aktywnego HotelEvent.
3	tick_process_reception_queue	Finalizacja zadań CheckIn / CheckOut, dla których upłynął remaining_task_minutes.
4	tick_process_failures_and_random_events	Obsługa już aktywnej awarii (FixFailure) lub wywołanie EventGenerator → ewentualnie nowe zdarzenie.
5	tick_evaluate_hotel_cancellations	Detekcja no-show meldunku — 120 min od planowanego check-in bez zameldowania.
6	tick_process_pending_cancellations	Wywołanie cancel_by_hotel() na rezerwacjach zakwalifikowanych w kroku 5.
7	tick_mark_daily_dirty_rooms	Raz na dobę: ustawienie to_clean = true dla pokoi zajętych.
8	tick_schedule_reception_tasks	Planowanie meldunków i wymeldowań na bieżący moment (z uwzględnieniem ewakuacji).
9	tick_process_meals	Trzyfazowa obsługa śniadania i kolacji (planowanie / gotowanie / serwowanie).
10	tick_dispatch_housekeeping	Przydział pokojówek do pokoi z to_clean (blokowane przy actual_failure).
11	tick_log_time_step	Wpis kategorii time do SimEventLog + ewentualny wydruk konsolowy (throttling).
12	refresh_hotel_report	Aktualizacja struktury HotelReport (saldo, średnia satysfakcji, etykieta jakości).

Metoda run(max_ticks) wywołuje powyższy ciąg w pętli. Dla celów demonstracyjnych pomiędzy tickami zastosowano std::this_thread::sleep_for(200 ms), co pozwala obserwować przebieg w konsoli w tempie zbliżonym do rzeczywistego. Po zakończeniu pętli wywołane są publish_hotel_report() (publikacja linii REPORT) oraz — jeśli skonfigurowano log_file — export_log_file() (zrzut zgromadzonych zdarzeń do pliku).

4.2. Modularyzacja kodu

Ze względu na rozmiar logiki biznesowej, implementacja klasy Simulation została podzielona na kilka plików tematycznych. Każdy moduł rozwija metody prywatne klasy Simulation we własnym zakresie odpowiedzialności, dzięki czemu plik nagłówkowy Simulation.hpp pozostaje spójny, a poszczególne aspekty są łatwe do testowania w izolacji.

Tabela 10. Podział modułów silnika symulacji

Moduł	Odpowiedzialność
Simulation.cpp	Konstruktor, run(), tick() — orkiestracja etapów, log_event, throttling
SimulationEvents.cpp	EventGenerator, PowerOutage / FireAlarm, no-show, awarie, cancel_by_hotel
SimulationMeals.cpp	Trzyfazowe planowanie posiłków, batch do 10, anulowania niedostarczenia
SimulationReception.cpp	Kolejki CheckIn / CheckOut, planowanie zadań recepcji
SimulationHousekeeping.cpp	Dyspozycja sprzątania pokoi (blokada przy awariach)
SimulationTime.cpp	Wirtualny zegar, kalendarz, formatowanie dat
simulation_rng.cpp	Wspólny generator pseudolosowy std::mt19937 dla całego projektu

4.3. Wzorce projektowe

Tabela 11. Wzorce projektowe zastosowane w projekcie

Wzorzec	Zastosowanie w projekcie
Polimorfizm dynamiczny	Room, Staff, Guest, HotelEvent — interfejsy wirtualne z dispatchem przez wskaźnik na klasę bazową
Fabryka statyczna	Task::make_check_in(), Task::make_cook_meal() i analogiczne — enkapsulacja losowania czasu
Kompozycja	Hotel agreguje kolekcje unique_ptr<Room/Staff/Guest> — brak współdzielonego stanu
Maszyna stanów	Cykl posiłku: planowanie → gotowanie → serwowanie; stany pokoju przez to_clean / to_repair
Konfiguracja zewnętrzna	HotelLoader, StaffLoader, ConfigLoader — parsowanie plików tekstowych z sekcjami
Single Responsibility (moduł)	Podział Simulation na pliki Simulation*.cpp wg domeny (recepcja / posiłki / sprzątanie)

Zarządzanie pamięcią opiera się konsekwentnie na std::unique_ptr — wszystkie obiekty domenowe (pokoje, pracownicy, goście) są własnością Hotel, a relacje są reprezentowane przez surowe wskaźniki obserwujące (Bill → Reservation) lub kompozycję.

5. System logowania i raportowania

5.1. Format wpisów i kategorie zdarzeń

Pojedynczy wpis logu w konsoli ma stałą strukturę: znacznik czasu wirtualnego, identyfikator kategorii w nawiasach kwadratowych, komunikat operacyjny w języku angielskim, opcjonalne pola kontekstowe w nawiasach okrągłych.

Listing 1. Format wpisu logu na konsoli

```
[YYYY-MM-DD HH:MM] [kategoria] komunikat (balance=..., weekday=...)
```

Identyfikatory kategorii są stałymi wewnętrznymi systemu i pozostają w języku angielskim, niezależnie od języka komunikatów raportu. Zestaw obecnych w wersji końcowej kategorii zdefiniowano w SimEventLog.hpp:

Tabela 12. Kategorie zdarzeń w systemie logowania

Kategoria	Znaczenie
time	Postęp zegara wirtualnego — emitowana w każdym ticku (objęta throttlingiem)
housekeeping	Sprzątanie pokoi — przypisanie, zakończenie, brak pokojówki
check_in	Zameldowania gości w recepcji
checkout	Wymeldowania gości w recepcji
failure	Awarie infrastruktury — PowerOutage, FireAlarm, ewakuacja, naprawa
simulation	Komunikaty meta — start/stop, parametry, fatal errors
restaurant	System gastronomiczny — planowanie, gotowanie, serwowanie, brak personelu
reservation	Cykl rezerwacji — utworzenie, anulowanie, no-show, cancel_by_hotel

Komunikaty operacyjne (np. „Checked in room 104”, „No available waiter”) celowo pozostawiono w języku angielskim, aby zachować spójność z konwencją projektową kursu PROI i ułatwić ewentualne ponowne wykorzystanie kodu w środowisku międzynarodowym.

5.2. Mechanizm log_every_n_ticks — warstwa konsolowa i pamięciowa

Wbrew sugestii w pierwotnej wersji raportu, log_every_n_ticks jest mechanizmem w pełni zaimplementowanym i pokrytym osobnym testem jednostkowym (tests/test_event_log.cpp). Konstrukcja jest dwuwarstwowa, co rozwiązuje typowy dylemat: szczegółowy log do analizy post-mortem versus czytelny strumień konsoli w czasie demonstracji.

5.2.1. Warstwa 1 — SimEventLog (pełna historia)

Metoda Simulation::log_event(category, message) ZAWSZE zapisuje wpis do struktury event_log_[category] niezależnie od ustawienia log_every_n_ticks. Oznacza to, że pełna historia każdego ticka — w tym kategoria time — pozostaje dostępna w pamięci dla późniejszej analizy lub eksportu. Throttling nie powoduje utraty informacji.

5.2.2. Warstwa 2 — konsola (selektywny throttling)

Decyzję o wyświetleniu wpisu w konsoli podejmuje predykat should_print_to_console(category):

- Jeśli category ≠ "time": wpis trafia na konsolę natychmiast. Wszystkie zdarzenia operacyjne (zameldowania, awarie, posiłki) są widoczne w czasie rzeczywistym.
- Jeśli category == "time" oraz current_tick % log_every_n_ticks == 0: wpis trafia na konsolę. Pozostałe ticki kategorii time są zapisywane wyłącznie do SimEventLog.
- Jeśli log_every_n_ticks ≤ 1: każdy tick kategorii time jest wyświetlany — efektywne wyłączenie throttlingu.

Komunikat kategorii time generowany w tick_log_time_step ma dwa warianty: w ticku spełniającym warunek N | current_tick wyświetlany jest pełny opis postępu („N ticks elapsed (N × minutes_per_tick min). Current tick: X"), w pozostałych — krótka forma „Tick X" zapisana wyłącznie do SimEventLog.

5.2.3. Konfiguracja i CLI

Parametr może być ustawiony na trzy sposoby:

- w pliku konfiguracyjnym sekcja [simulation]: log_every_n_ticks=5,
- przez flagę CLI: --log-every-n-ticks N (parsowana w ConfigLoader, nadpisuje plik),
- brak ustawienia: wartość domyślna 1 (każdy tick na konsoli).

5.2.4. Test jednostkowy

Test tests/test_event_log.cpp, scenariusz „log_every_n_ticks throttles console but keeps full event log", weryfikuje obie warstwy jednocześnie. Konfiguracja: 6 ticków, log_every_n_ticks = 2. Asercje:

- SimEventLog dla kategorii time zawiera 6 wpisów (po jednym na każdy tick),
- konsola otrzymała wyłącznie ticki spełniające warunek modulo (tj. 2, 4, 6).

5.2.5. Przykład rzeczywistego logu

Poniżej fragment logu wygenerowany w scenariuszu Hotel Grand Aurora z log_every_n_ticks = 5, ilustrujący współistnienie obu warstw:

```
[2026-06-02 08:25] [time] 5 ticks elapsed (25 min). Current tick: 5
[2026-06-02 08:27] [check_in] Checked in room 104 (balance=12350)
[2026-06-02 08:30] [restaurant] Cooking breakfast batch (8 meals)
[2026-06-02 08:50] [time] 5 ticks elapsed (25 min). Current tick: 10
[2026-06-02 09:12] [failure] PowerOutage triggered — actual_failure=true
```

Wiersz [time] występuje co 5 ticków (co 25 minut wirtualnych), wszystkie pozostałe kategorie są wyświetlane natychmiast w momencie wystąpienia zdarzenia.

5.3. Eksport do pliku — Logger::prepare_log

Po zakończeniu run() wywoływana jest metoda export_log_file(), która deleguje formatowanie i zapis do klasy Logger (src/logger/Logger.cpp). Eksport realizowany jest w następującej sekwencji:

- Logger::prepare_log oblicza efektywny interwał: file_interval = log_file_every_n_ticks (jeśli > 0) LUB log_every_n_ticks (wartość zapasowa).
- Logger iteruje ticki od 0 do current_tick co file_interval; dla każdego ticka wybiera WSZYSTKIE wpisy ze wszystkich kategorii z SimEventLog (nie tylko kategorię time).
- Wyszukiwanie wpisów odpowiadających danemu tickowi realizowane jest przez std::lower_bound — wiele zdarzeń występujących w jednym ticku jest obsłużonych w pętli while iterującej po wszystkich wpisach o danym current_tick.
- Wynikowe linie zapisywane są do pliku wskazanego parametrem log_file (domyślnie hotel_simulation.log).

Konstrukcja zapewnia, że plik wynikowy zawiera komplet zdarzeń (a nie wyłącznie ticki kategorii time) — jest zatem materiałem nadającym się do analizy post-mortem nawet wtedy, gdy konsola była mocno wyciszona.

5.4. Raport końcowy i progi oceny

Struktura HotelReport, aktualizowana w każdym ticku przez refresh_hotel_report(), agreguje cztery wskaźniki: bieżące saldo hotelu, liczbę zameldowanych gości, średnią satysfakcję oraz etykietę jakości usług. Etykieta wyliczana jest na podstawie progów:

Tabela 13. Progi oceny jakości usług

Średnia satysfakcja	Etykieta	Interpretacja
≥ 80	excellent	Brak istotnych zastrzeżeń, hotel działa wzorcowo
≥ 60	good	Drobne incydenty obsłużone, gość zadowolony
≥ 40	average	Zauważalne braki w obsłudze
< 40	poor	Poważne braki — kompromituje markę hotelu

6. Zdarzenia losowe i obsługa awarii

Tabela 14. Stan realizacji zdarzeń losowych

Zdarzenie	Status	Opis funkcjonalny
PowerOutage	Zrealizowane	Wyłączenie zasilania hotelu, ustawia actual_failure = true
FireAlarm	Zrealizowane	Tryb ewakuacji — wstrzymanie operacji recepcji, set_evacuation(true)
RoomBreakdown	Zastąpione actual_failure	Funkcję pełni flaga Hotel::actual_failure + zadanie FixFailure
LostKey	Niezrealizowane	Brak implementacji w wersji końcowej — odłożone

6.1. EventGenerator — losowanie i typy zdarzeń

Klasa EventGenerator (src/events/EventGenerator.cpp) jest wywoływana w każdym ticku w ramach etapu tick_process_failures_and_random_events. Wykorzystuje wspólny generator simulation_rng (ujednolicony z resztą projektu — wpis „Ujednolicenie EventGenerator z simulation_rng" z pierwotnej listy TODO został zrealizowany, a komentarz TODO usunięty z kodu).

Mechanika losowania:

- Parametr event_probability wyrażony jest w skali 1–1000 (setne procenta): wartość 5 = 0,5 %, 50 = 5 %, max 300 = 30 %.
- EventGenerator losuje liczbę całkowitą z przedziału 1–1000 (uniform_int_distribution).
- Jeśli wylosowana wartość ≤ event_probability, generowane jest zdarzenie. Wybór typu: 70 % szans na PowerOutage, 30 % na FireAlarm.

Istotne: EventGenerator NIE produkuje zdarzeń typu RoomBreakdown jako osobnej klasy HotelEvent. Funkcję awarii infrastrukturalnej pełnią PowerOutage i FireAlarm, które poprzez side-effect ustawiają actual_failure (patrz 6.2).

6.2. actual_failure jako odpowiednik RoomBreakdown

W dokumentacji wstępnej (diagram UML) przewidziano osobną klasę HotelEvent o nazwie RoomBreakdown, reprezentującą awarię konkretnego pokoju. W wersji końcowej z uwagi na uwagi prowadzącego (redukcja zakresu) zrezygnowano z odrębnej klasy, a jej funkcję zrealizowano przez flagę Hotel::actual_failure oraz dedykowane zadanie pracy konserwatora FixFailure:

- PowerOutage::apply() i FireAlarm::apply() — obie metody ustawiają actual_failure = true,
- Hotel::try_fix_failure() — wywołuje try_assign_maintenance(), które poprzez assign_first_available_staff przydziela MaintenanceWorkera do zadania Task::make_fix_failure(),
- Room::to_repair / Room::destroy() — istnieją w kodzie i są pokryte testami, ale w bieżącej wersji NIE są spięte z EventGenerator (pole pomocnicze, opcjonalnie ustawiane w hotel.txt parametrem to_repair=true do testów ręcznych).

Konstrukcja oznacza, że — z punktu widzenia symulacji — awaria jest stanem globalnym hotelu, a nie cechą pokoju. Decyzja ta jest świadomym uproszczeniem względem UML i została omówiona z prowadzącym.

6.3. Zachowanie alarmu pożarowego (FireAlarm)

Aktywacja FireAlarm wprowadza hotel w tryb ewakuacji o szerokim zasięgu działania. Sekwencja efektów w metodzie FireAlarm::apply():

- set_evacuation(true) — flaga globalna wstrzymująca operacje obsługi,
- set_power(false) — wyłączenie zasilania hotelu (powiązany efekt PowerOutage),
- actual_failure = true — kwalifikacja do naprawy przez MaintenanceWorkera.

Konsekwencje dla pozostałych modułów:

- Recepcja: na początku schedule_check_in_and_check_out wykonywany jest return — żadne meldunki ani wymeldowania nie są planowane w czasie ewakuacji.
- System posiłków: planowanie i serwowanie jest wstrzymane przy actual_failure lub evacuation.
- Sprzątanie: tick_dispatch_housekeeping pomija nowe pokoje, jeśli actual_failure jest aktywne.

Po pomyślnym ukończeniu zadania FixFailure metoda clear() na aktywnym zdarzeniu przywraca stan normalny: wyłącza ewakuację, przywraca zasilanie i czyści actual_failure.

6.4. Warunek krytyczny awarii i no-show meldunku

Aby zapobiec sytuacji, w której awaria pozostaje nienaprawiana w nieskończoność pomimo dostępnego konserwatora (potencjalny błąd logiki przydziału), wprowadzono warunek krytyczny: nienaprawiona awaria trwająca 40 minut wirtualnych przy dostępnym pracowniku Maintenance powoduje rzucenie wyjątku z komunikatem „Hotel has been destroyed”. Mechanizm pełni funkcję sanity-check w czasie testów i wymusza priorytetowe traktowanie usterek infrastruktury.

Analogiczny mechanizm graniczny stosowany jest dla no-show meldunku, ale z innym efektem. Detekcja w tick_evaluate_hotel_cancellations zachodzi, gdy:

- rezerwacja jest na dzisiaj, gość nie został jeszcze zameldowany,
- upłynęło 120 minut od planowanego czasu check-in,
- hotel NIE jest w stanie evacuation ani actual_failure (no-show jest wstrzymane w czasie awarii — wina hotelu, nie gościa).

Następnie tick_process_pending_cancellations wywołuje cancel_by_hotel(), które ustawia cancelled = true, is_paid = true (gość nie zostaje obciążony) oraz — jeśli do daty check-in pozostaje mniej niż 1 dzień — redukuje średnią satysfakcję, traktując sytuację jako wynik niewydolności obsługi recepcji.

7. System obsługi gastronomicznej

System posiłków stanowi rozszerzenie funkcjonalne wykraczające poza pierwotną dokumentację wstępną. Realizuje trzyfazowy proces obsługi śniadania i kolacji, w pełni zintegrowany z mechaniką personelu, satysfakcji gości i awarii.

Trzy fazy cyklu posiłku

- **Planowanie.** Na początku okna posiłkowego (śniadanie / kolacja) wyliczana jest lista gości partycypujących. Spośród rezerwacji z atrybutem food_included == true losowany jest podzbiór o liczności meal_attendance_percent % (domyślnie 80 %). Pozostali goście traktowani są jako nieobecni w restauracji — bez kary satysfakcji, ale bez przychodu z posiłku.
- **Gotowanie.** Liczba zaplanowanych posiłków dzielona jest na batche o rozmiarze do 10 — pojedyncze zadanie CookMeal obsługuje jeden batch. Hotel::assign_first_available_staff szuka Chefa zgodnie z algorytmem 3.5.3 (rola Chef, fatigue < 100, nie zajęty, can_handle). Brak dostępnego kucharza skutkuje wpisem „No available chef" w kategorii restaurant oraz redukcją satysfakcji wszystkich gości w batchu.
- **Serwowanie.** Po zakończeniu CookMeal kelner (rola Waiter) zostaje przypisany do zadania ServeMeal — dostarcza posiłek do konkretnego pokoju. Brak kelnera generuje wpis „No available waiter" oraz redukcję satysfakcji adresata. We wcześniejszej wersji konfiguracji z niedoborem kelnerów to właśnie ta kategoria zdarzeń odpowiadała za wynik balance = -2573 / rating = poor.

Anulowania i interakcje z awariami

Logika no-show posiłku jest istotnie różna od no-show meldunku. Goście, którzy nie pojawili się na konkretnym posiłku, są ewidencjonowani w meal_no_shows_, ale ich rezerwacje NIE są anulowane — gość pozostaje w hotelu i może spożywać kolejne posiłki. Nieskonsumowane posiłki nie obciążają bilansu hotelu (nie zostały ugotowane).

System posiłków jest wstrzymywany w pełni przy actual_failure lub evacuation — żadne nowe gotowanie ani serwowanie nie startuje w trybie awarii. Aktywne zadania CookMeal i ServeMeal kończą się normalnie (nie są przerywane), co modeluje realistyczne zachowanie: kucharz dokończy aktualny batch, ale nie zacznie kolejnego.

Wszystkie zdarzenia gastronomiczne (rozpoczęcie gotowania, brak personelu, anulowanie no-show) są zapisywane w SimEventLog w kategorii restaurant — nie są więc filtrowane przez throttling kategorii time.

8. Konfiguracja systemu

Parametry hotelu, personelu i symulacji są definiowane w plikach tekstowych o strukturze sekcji oznaczonych nagłówkami w nawiasach kwadratowych: [simulation], [room], [worker], [reservation]. Ładowanie realizują trzy współpracujące klasy: HotelLoader (pokoje i rezerwacje), StaffLoader (personel z walidacją harmonogramów), ConfigLoader (parametry symulacji i parsowanie CLI). Wybrane parametry można nadpisać z linii poleceń, co ma pierwszeństwo nad plikiem konfiguracyjnym.

Tabela 15. Wybrane parametry konfiguracyjne

Klucz	Opis
minutes_per_tick	Liczba minut symulowanych w jednym ticku (typowo 5)
max_ticks	Długość symulacji; 0 = jedna doba
event_probability	Szansa zdarzenia na tick w skali setnych procenta (5 = 0,5 %, max 300 = 30 %)
date_start	Data początkowa symulacji w formacie YYYY-MM-DD
log_every_n_ticks	Interwał wyświetlania kategorii time na konsoli
log_file_every_n_ticks	Opcjonalny niezależny interwał dla pliku logu (jeśli > 0)
log_file	Ścieżka eksportu logu po zakończeniu run()
meal_attendance_percent	Procent gości partycypujących w posiłku (domyślnie 80)

Tabela 16. Wybrane flagi CLI (parsowane w ConfigLoader)

Flaga CLI	Efekt
--hotel <ścieżka>	Wskazuje plik konfiguracyjny hotelu (wymagana)
--max-ticks N	Nadpisuje max_ticks z pliku
--log-every-n-ticks N	Nadpisuje log_every_n_ticks
--log-file <ścieżka>	Nadpisuje log_file

9. Weryfikacja poprawności

Poprawność implementacji weryfikowana jest zestawem szesnastu plików testów jednostkowych zbudowanych na frameworku Catch2/doctest. Każdy plik koncentruje się na pojedynczym obszarze odpowiedzialności i posiada od kilku do kilkunastu osobnych scenariuszy testowych.

Tabela 17. Pokrycie testami jednostkowymi

Plik testu	Weryfikowany obszar
test_guests.cpp	Mechanika satysfakcji per typ gościa (mnożniki ×1 / ×3 / ×5)
test_staff.cpp	Role, walidacja harmonogramów, accrue_shift_pay, fatigue, rest_tick

test_rooms.cpp	Ceny (calculateNightlyPrice), stany to_clean / to_repair, przejścia
test_housekeeping.cpp	Przypisywanie pokojówek do pokoi wymagających sprzątania
test_maintenance.cpp	Przypisanie konserwatora, naprawa actual_failure, opłata awaryjna 500
test_bill.cpp	Kalkulacja rachunków: noclegi + opcjonalne wyżywienie
test_meal_dispatch.cpp	Trzyfazowa dystrybucja posiłków, batche do 10
test_events.cpp	EventGenerator: prawdopodobieństwo, podział 70/30 PowerOutage/FireAlarm
test_hotel_event.cpp	PowerOutage::apply, FireAlarm::apply, clear()
test_reservation.cpp	Cykl rezerwacji: utworzenie, anulowanie, cancel_by_hotel, no-show
test_logger.cpp	Logger::prepare_log, eksport do pliku z file_interval
test_event_log.cpp	Throttling log_every_n_ticks: konsola vs SimEventLog
test_config_loader.cpp	Parsowanie sekcji, walidacja kluczy, parsowanie CLI
test_simulation_date.cpp	Kalendarz wirtualny, formatowanie dat
test_staff_loader.cpp	Ładowanie personelu z pliku, walidacja zmian
test_hotel_loader.cpp	Ładowanie pokoi i rezerwacji

Uruchomienie pełnego zestawu testów odbywa się jednym poleceniem ctest --test-dir build. W wersji końcowej wszystkie testy przechodzą pomyślnie.

10. Podział prac w zespole

10.1. Bartosz Grzanka

- Projekt i implementacja silnika Simulation oraz modułów tematycznych Simulation*.cpp.
- Mechanika personelu: hierarchia Staff i klasy pochodne, klasa Task z fabrykami make_*, model zmęczenia, naliczanie wynagrodzeń per-tick, walidacja harmonogramów.
- System logowania: dwuwarstwowy throttling konsola vs SimEventLog, eksport przez Logger::prepare_log z wyszukiwaniem binarnym po ticku.
- Ładowanie konfiguracji: ConfigLoader (sekcje + CLI), HotelLoader, StaffLoader.
- Scenariusz demonstracyjny config/hotel.txt — 18 pokoi, 19 pracowników, 3 doby symulacji.

10.2. Konrad Orlikowski

- Hierarchia pokoi: klasa abstrakcyjna Room i klasy pochodne SingleRoom, DoubleRoom, TripleRoom, Apartment, logika stanów to_clean / to_repair, calculateNightlyPrice i calculateStaffNeeded.

- Obsługa gości: hierarchia Guest (Family / Business / VIP z mnożnikami satysfakcji), Reservation, Bill, HotelReport z progami oceny.
- System zdarzeń: EventGenerator z ujednoliconym simulation_rng, klasy HotelEvent — PowerOutage, FireAlarm.
- Struktury SimEventLog i wyszukiwanie binarne po current_tick (lower_bound) wykorzystywane przez Logger.

10.3. Prace wspólne

- Testy jednostkowe (16 plików) i ich integracja z CTest.
- Weryfikacja scenariusza demonstracyjnego Hotel Grand Aurora — strojenie obsady personelu po analizie logów no-show.
- Uzgodnienie interfejsów pomiędzy klasami domenowymi i klasą Simulation (eliminacja behave()).
- Przygotowanie dokumentacji końcowej i diagramów.

11. Ograniczenia i kierunki rozwoju

Wersja końcowa świadomie zrezygnowała z części elementów planu wstępnego — głównie w odpowiedzi na uwagi prowadzącej dotyczące zbyt szerokiego zakresu projektu. Pozostałe ograniczenia wynikają z dostępnego czasu lub świadomych decyzji projektowych.

Tabela 18. Ograniczenia i zaplanowane rozszerzenia

Element	Status w wersji końcowej
LostKey jako osobne HotelEvent	Niezaimplementowane — poza zakresem czasowym projektu
RoomBreakdown jako osobna klasa HotelEvent	Świadomie zastąpione flagą actual_failure i zadaniem FixFailure
Klasa Studio z diagramu UML	Zastąpione TripleRoom / Apartment
Szczegółowe udogodnienia pokoi (łazienki, Wi-Fi, balkon)	Poza zakresem wersji końcowej
Ujednolicenie EventGenerator z simulation_rng	Zrealizowane (TODO usunięte z kodu)
Powiązanie Room::to_repair z EventGenerator	Pole pomocnicze testowe, niepołączone z generatorem

Naturalne kierunki dalszego rozwoju projektu obejmowałyby przywrócenie zakresu z UML (LostKey, RoomBreakdown per pokój), wprowadzenie heterogeniczności pokoi (udogodnienia jako modyfikatory ceny i satysfakcji), a także rozbudowę modelu finansowego o koszty stałe (utrzymanie pokojów niezajętych, amortyzacja sprzętu).